ANTHROP\C

Audited with Claude Mythos 5.1

Security audit of a post-quantum storage plugin.

How Claude Mythos 5.1 read the code, hunted for bugs, checked its own findings and ran experiments. With every result.

TARGET	capacitor-pq-secure-storage, a Capacitor plugin for iOS, Android and web
REPOSITORY	github.com/jimcase/capacitor-pq-secure-storage
CODE AUDITED	Branch 7.x, commit 8d9a0f4, no tracked changes
MODEL	claude-mythos-5-1, interactive session in Claude Code
FIXED IN	7.1.0 (Capacitor 7 line) and 8.1.0 (Capacitor 8 line)
DATE	2026-10-30
AUTHOR	J Caso

18
new findings in code reviewed 11 times

3
proven with real code or dependency source

18
fixed and released as 7.1.0 and 8.1.0

Independent audit of the author's own open-source plugin, run with Claude Mythos 5.1. Claude, Mythos and the Anthropic logo are trademarks of Anthropic, PBC. This document is not affiliated with, sponsored or endorsed by Anthropic.

SUMMARY

What we did and what we found.

capacitor-pq-secure-storage keeps post-quantum keys and secrets in platform hardware: ML-DSA signatures, ML-KEM encryption and a biometric secure store. Before this audit it had been through 11 review rounds, with about 50 findings fixed.

We asked Claude Mythos 5.1 to audit it again. The model read all native code and the supporting files. It hunted with five focus areas, tried to prove each of its own findings wrong, and ran experiments on this machine wherever it could.

18
new findings: 3 high, 5 medium, 10 low

3
proven: two executed, one checked in dependency source

8
entries of the old catalog were incomplete or wrong

What matters most

- **M-3.** The ML-KEM secret on Android is not the same as on iOS and web. Data does not open across platforms. Executed and proven.
- **M-4.** One invalid base64 string from the webview closes the Android app. A regression of an earlier fix.
- **M-5.** The overwrite gate never worked for ECDSA keys on Android. The alias is lost for good. Executed and proven.
- **M-1 and M-2.** Two ways around the biometric gate on Android: a tier downgrade and a wipe without a prompt.
- **M-8.** A possible biometric bypass on iOS. Only plausible until a real iPhone answers.

What it could not do

No real device was available during the audit, so nothing was tested on a Secure Enclave or on KeyMint. Eight device checks are still open. All 18 findings were fixed afterwards and released as 7.1.0 and 8.1.0. The page on fixes shows how each one was checked.

How to read this report

Scope and threat model, then the method, then the experiments. After that, the results, the findings in detail, what was dropped, what is still open, and a playbook to repeat the process on other code.

SCOPE

What was audited, and against whom.

AREA	FILES READ
iOS	PqSecureStoragePlugin.swift (1,444 lines), tests, Package.swift, podspec
Android	PqSecureStoragePlugin.kt (1,453 lines), unit and instrumented tests, build.gradle, variables.gradle, proguard rules, manifest
TypeScript	definitions, web fallback, enhance, json, algorithms, index (about 700 lines), package.json
Tests and docs	45 JS tests with a software double, README, DEVICE-VERIFICATION, CI workflow
History	The July catalog: about 50 earlier findings with fix, commit and status

Threat model

Given to the model before it read a line of code. These five attackers are what counts.

1 **Hostile JavaScript in the webview**
Any arguments, any order, in parallel, too big or malformed.

2 **Unlocked phone, no biometrics**
Tries to read, overwrite, delete or downgrade biometric items.

3 **Backup, root or another app**
Tries to read items or swap keys for its own.

4 **Crash or re-enrollment mid-operation**
Half writes, power loss, a new fingerprint. Is the store left broken?

5 **Platform confusion**
A guarantee on iOS that Android lacks, with no flag to say so.

Out of scope

A jailbreak with a hooked LAContext, a full snapshot rollback by root, bugs inside the OS or BouncyCastle, and every limit already documented in the project. Real-device runs, dynamic tests of host apps and supply chain beyond the version pins were also left out.

Severity scale

HIGH bypass of the biometric gate, tier downgrade, persistent brick. MEDIUM denial of service, inconsistency the user can recover from, leak of item names. LOW hardening. A critical level exists for key disclosure. None was found.

METHOD

Four steps, in this order.

1 Give it context and strict rules

The code was pinned first: branch 7.x, commit 8d9a0f4, no tracked changes. The model got the source, tests, docs, CI, build files and the July catalog. The catalog lets it avoid known items and check that old fixes still hold.

Rules for every finding: file and line with the code quoted. A concrete attack: who does what, in what order, and what they get. The fix in one or two sentences. Severity and an honest confidence. No style notes. No documented limits. Prefer five findings you are sure of over twenty you are not.

2 Hunt with five lenses

- **iOS:** Keychain access control, Secure Enclave use, LAContext, atomic writes, error paths.
- **Android:** Keystore parameters, BiometricPrompt and CryptoObject binding, AES-GCM at rest, BouncyCastle ML-KEM, atomicity.
- **Bridge:** the TypeScript layer, argument validation, what the web fallback promises, what hostile JS can do.
- **Crypto:** keys, nonces, AAD, encodings, signature inputs. Do the three platforms agree?
- **Regression:** walk the catalog section by section and check that each fix is still in the code.

Each lens finds different things. The crypto lens found M-3. The regression lens found M-4 and M-5.

3 Make it kill its own findings

A second pass reads the code around each finding, traces every path that reaches it, and gives a verdict: **CONFIRMED** if the scenario works as described, **PLAUSIBLE** if it is likely real but depends on device behavior, **REJECTED** if it is unreachable, already mitigated, or a known limit. About a dozen suspects were dropped here. They are listed later in this report.

4 Run what can be run

Where a claim could be tested on this machine, the model wrote a small experiment and kept the output. Three experiments are on the next page. Claims that need a real device stay PLAUSIBLE, with the exact check to do.

> **The whole run happened in Claude Code, with the repository and a shell at hand. The same method can be scripted against the API in batches.**

EXPERIMENTS

Claims that were run, not just argued.

1. BouncyCastle against noble (M-3)

Procedure: BouncyCastle 1.85 was downloaded from Maven Central and its sha256 checked against the published file. A short Java program generated an ML-KEM-768 key pair. Node, with @noble/post-quantum 0.6.1 (the version the plugin pins), encapsulated to that public key. The Java program then decapsulated with the same call the plugin makes.

```
plugin call, KEMExtractSpec(priv, ct, "AES"):
  f46b31886a143eb750e828d8c833c32d8c9b9d59375cb0b2a929a8539a768173
noble shared secret (raw K):
  94b8608b73b00f35c99d0f5daabcdc7cca0fd7fa4e207d64c41424bdd4afda7b
MATCH_DEFAULT=false

BouncyCastle with withNoKdf():
  94b8608b73b00f35c99d0f5daabcdc7cca0fd7fa4e207d64c41424bdd4afda7b
MATCH_NOKDF=true
```

The bc-java source confirms the cause: the two-argument constructor uses DefKdf, which is KDF3 with SHA-256, and Builder.withNoKdf() removes it. The same test on the encapsulation side also matched only with withNoKdf().

2. One line on the JVM (M-5)

```
Signature.getInstance("EC")
-> NoSuchAlgorithmException: EC Signature not available   (JDK 24)
```

The plugin builds its signer from the key algorithm name. For an EC key from the Keystore that name is "EC", so the biometric gate for overwriting ECDSA keys fails every time.

3. Read the dependency (M-4)

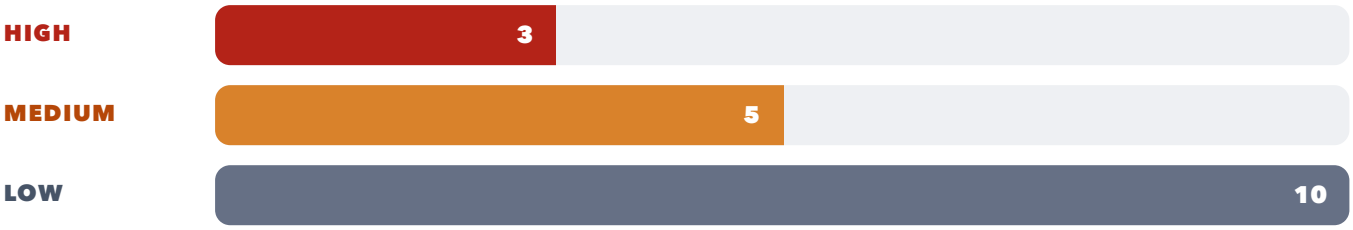
Capacitor Android 8.4.1, Bridge.java lines 839 to 851: an exception that escapes a plugin method is logged and rethrown as a RuntimeException on the bridge thread. The code is the same on the Capacitor 7 line.

Baseline

The existing suite, run before and after reading: `npx vitest run` gave 5 files and 45 tests, all green. Nothing in the plugin was changed during the audit. After the fixes the suites run 52 JS tests, 30 JVM tests and 15 iOS tests.

RESULTS

18 new findings.



One of the three high findings, M-8, is only plausible until a real iPhone confirms it.

How each finding was verified

METHOD	COUNT	FINDINGS
EXECUTED	2	M-3, M-5
DEPENDENCY SOURCE	1	M-4
CODE READING	14	M-1, M-2, M-6, M-7, M-9 to M-18
NEEDS DEVICE	1	M-8

Earlier fixes that did not hold

The regression lens compared the code with the July catalog. Eight entries turned out incomplete or wrong.

CATALOG ITEM	WHAT HAPPENS NOW	FINDING
Overwrite gate (A-1, A-2)	Never worked for ECDSA keys on Android	**M-5**
No KDF on ML-KEM (B-8)	False premise: Android applies KDF3	**M-3**
Tier with invalid key (C-6)	Covers the dead key, not the orphan key	**M-1**
Wrap key rotation (D-2, D-3)	Done for KEM, not for the Ed25519 path added later	**M-6**
Input limits (E-4, E-10)	iOS counts characters, not bytes	**M-12**
Cap before decoding (E-11)	Ended up outside the try block and opened a crash	**M-4**
Biometric flag (G-5)	Fixed on Android, not on iOS	**M-9**
Listing with UISkip (F-1, F-3)	Still used in clear() and keys()	**M-8**

ALL FINDINGS

The 18 findings at a glance

Plugin.kt is android/src/main/java/com/capacitorpqsecurestorage/PqSecureStoragePlugin.kt. Plugin.swift is ios/Sources/PqSecureStoragePlugin/PqSecureStoragePlugin.swift. HIGH? means high if confirmed.

ID	SEV.	PLATFORM	LOCATION	FINDING	CHECK
M-1	HIGH	Android	`Plugin.kt:1212–1227`	setItem reuses an orphan key with the wrong tier	CODE READING
M-2	HIGH	Android	`Plugin.kt:1407–1435`	clear() wipes with no prompt when the first biometric key is invalidated	CODE READING
M-3	MEDIUM	Android	`Plugin.kt:1275, 1299`	ML-KEM secret on Android is not the raw K	EXECUTED
M-4	MEDIUM	Android	`Plugin.kt:494, 539, 745`	Invalid base64 crashes the host app	DEPENDENCY SOURCE
M-5	MEDIUM	Android	`Plugin.kt:430–453`	Overwrite gate fails for ECDSA keys, alias is bricked	EXECUTED
M-6	MEDIUM	Android	`Plugin.kt:108–188`	Ed25519 wrap key is retired before the commit	CODE READING
M-7	MEDIUM	Android + iOS	`Plugin.kt:111, 308, 461`	One alias can hold two keys	CODE READING
M-8	HIGH?	iOS	`Plugin.swift:1362–1369, 1400–1411`	clear() and keys() enumerate with UISkip	NEEDS DEVICE
M-9	LOW	iOS	`Plugin.swift:263–267`	biometricGated ignores enrollment	CODE READING
M-10	LOW	Android	`Plugin.kt:405–416, 955–995`	Silent keys usable while the device is locked	CODE READING
M-11	LOW	iOS	`Plugin.swift:411–422`	sign() prompts before validating	CODE READING
M-12	LOW	iOS	`Plugin.swift:1159–1161`	setItem limits count characters, not bytes	CODE READING
M-13	LOW	iOS + Android	`Plugin.swift:539–543`	recipientPublicKey has no cap before decoding	CODE READING
M-14	LOW	iOS	`Plugin.swift:757–786`	Secure Enclave signing key is created without a lock	CODE READING
M-15	LOW	Docs	`README.md:241–259`	Backup exclusion in the README is incomplete	CODE READING
M-16	LOW	TypeScript	`enhance.ts:20–37`	Key prefix is global state	CODE READING
M-17	LOW	Android	`Plugin.kt:216, 1049–1053`	Caller controls the prompt title on Ed25519	CODE READING
M-18	LOW	All	`Plugin.kt:1205, Plugin.swift:1177`	Non-boolean flags are treated as defaults	CODE READING

FINDING M-3 | MEDIUM | ANDROID | EXECUTED

Same key exchange, different secret.

Android encapsulates and decapsulates with BouncyCastle ML-KEM. The plugin builds the specs with the short constructors, so BouncyCastle applies its default KDF, KDF3 with SHA-256, to the shared secret. iOS (the Secure Enclave decapsulate) and the web fallback (noble) use the raw FIPS 203 secret.

```
// PqSecureStoragePlugin.kt, line 1275
kg.init(KEMGenerateSpec(pub, "AES"), SecureRandom())
// line 1299
kg.init(KEMExtractSpec(priv, kemCt, "AES"))
```

Effect

An encryptTo done on web, iOS, keripy or signify to the ML-KEM public key of an Android ends in a decrypt error, and the other way around. Android with Android works, because both sides apply the KDF. That is why the existing unit tests pass: they compare BouncyCastle with BouncyCastle.

This is not a weakness of the primitive. KDF3 of K is a good key. It is a broken promise between platforms, and a false premise in the old catalog, which said the secret was used directly with no KDF. The wallet does not consume the plugin yet, so no stored Android frames need a migration.

Fix

```
kg.init(KEMGenerateSpec.Builder(pub, "AES", 256).withNoKdf().build(), SecureRandom())
kg.init(KEMExtractSpec.Builder(priv, kemCt, "AES", 256).withNoKdf().build())
```

Test to add

Decapsulate a ciphertext made by another library and open a ChaCha20-Poly1305 frame sealed with the raw secret. With the current code the open step throws an authentication tag error. With withNoKdf() it passes. Test vectors from BouncyCastle 1.85 and noble 0.6.1 are saved next to this report.

FINDING M-4 | MEDIUM | ANDROID | DEPENDENCY SOURCE

One bad string closes the app.

Base64 decoding sits outside the try block in sign, encryptAtRest and encryptTo. android.util.Base64.decode throws on characters outside the alphabet.

```
// sign(), line 494 (the try starts at 497)
val input = decodeCapped(call, data) ?: return
// decodeCapped(), line 261
val bytes = Base64.decode(data, Base64.DEFAULT)

// Capacitor Bridge.java, lines 848 to 851
} catch (Exception ex) {
    Logger.error("Serious error executing plugin", ex);
    throw new RuntimeException(ex);
}
```

Attack: sign({ keyAlias: "a", data: "***", type: "PQC_MLDSA_65" }) from the webview. The process dies. It is a regression of an earlier fix (E-11), which moved the decoding in front of the try. Other Keystore calls outside a try can do the same on a transient failure.

Fix: catch the exception inside decodeCapped and reject with E_INVALID_ARGS. For the rest, wrap each plugin method in a small guard that rejects instead of throwing.

```
val bytes = try {
    Base64.decode(data, Base64.DEFAULT)
} catch (e: IllegalArgumentException) {
    call.reject("Invalid base64 input", "E_INVALID_ARGS"); return null
}
```

FINDING M-5 | MEDIUM | ANDROID | EXECUTED

The gate that never opens for EC keys.

To overwrite a biometric signing key, the plugin asks for a biometric bound to that key. It builds the signer from the key algorithm name.

```
// line 434
val signer = Signature.getInstance(priv.algorithm)
```

For an EC key from the Keystore that name is "EC", a KeyFactory name. On the JVM: NoSuchAlgorithmException. Overwriting a biometric ECDSA key always fails, the branch that allows an overwrite after an enrollment change is never reached, and there is no delete API. The alias is lost for good. Earlier fixes A-1 and A-2 never worked for EC on Android.

```
private fun signatureAlgoFor(priv: java.security.PrivateKey): String =
    if (priv.algorithm == "EC") "SHA256withECDSA" else priv.algorithm
```

FINDING M-1 | HIGH | ANDROID | CODE READING

A biometric item stored under a silent key.

```
// setItem, lines 1212 and 1216
if (existing != null && storeItems().contains(nameTag(key)) && isAuthRequired(existing) != newBio) {
... E_TIER_MISMATCH }
val itemKey: SecretKey = if (existing != null && !invalidated(existing)) {
    existing // reuse
} else { createItemKey(alias, newBio, ...) }
```

The tier check is skipped when no item exists for the name, which is what an orphan key from a failed write looks like. The next line then reuses that orphan without looking at its tier.

1. With the app in the background and the device locked, hostile JS calls setItem with the default accessibility. The key is created, the encryption fails, the key stays.
2. Later the host stores its seed with requireBiometric: true under that name.
3. The orphan silent key is reused. The user sees a biometric prompt and approves it.
4. From then on getItem reads the seed with no prompt.

Fix: reuse a key only when a real item is behind it, and recreate it with the requested tier otherwise. Confidence 0.75. The trigger needs a device check.

```
val hasItem = storeItems().contains(nameTag(key))
...
val itemKey = if (existing != null && hasItem && !invalidated(existing)) existing else
createItemKey(alias, newBio, ...)
```

FINDING M-2 | HIGH | ANDROID | CODE READING

clear() wipes everything with no prompt.

```
val bioAlias = itemAliases.firstOrNull { a -> (ks.getKey(a, null) as? SecretKey)?.let {
isAuthRequired(it) } ?: false }
...
} catch (e: KeyPermanentlyInvalidatedException) {
    doClearWipe()
    call.resolve()
```

After a biometric re-enrollment every old key is dead and nobody deletes it. If the host later stores a new biometric item and JS calls clear(), the alias order is arbitrary. When the first biometric key is a dead one, the invalidation exception is read as "nothing to protect" and everything is wiped, the live item included.

Fix: bind the prompt to a biometric key that is still alive, and keep the catch only for the race. Confidence 0.8.

```
val bioKey = bioKeys.firstOrNull { !invalidated(it) }
if (bioKey == null) { doClearWipe(); call.resolve(); return }
```

THE OTHER FINDINGS | 1 OF 2

M-6 to M-11

M-6 MEDIUM Ed25519 wrap key is retired before the commit

CODE READING

`Android | Plugin.kt:108–188`

A cancelled or failed replacement deletes the wrap key of the live alias. The old private key can no longer be opened and sign() fails forever.

Fix: Version the wrap key, as the KEM path already does. Confirmed by reading the code.

M-7 MEDIUM One alias can hold two keys

CODE READING

`Android + iOS | Plugin.kt:111, 308, 461`

The "alias exists" check does not cross the hardware and wrapped namespaces. JS can create a silent Ed25519 key under an identity alias, and getPublicKey returns it.

Fix: Check both namespaces before creating. Confirmed by reading the code. The web fallback already rejects it.

M-8 HIGH? clear() and keys() enumerate with UISkip

NEEDS DEVICE

`iOS | Plugin.swift:1362–1369, 1400–1411`

If the system also hides ACL-protected items in attribute queries, clear() sees only silent items and deletes the biometric ones without a prompt.

Fix: List with UIFail and treat InteractionNotAllowed as "a biometric item exists". Plausible. Needs a real device. Confidence 0.3.

M-9 LOW biometricGated ignores enrollment

CODE READING

`iOS | Plugin.swift:263–267`

It means "Secure Enclave present", not "biometrics enrolled". Hosts get a wrong answer and later calls fail with no clear reason.

Fix: Use canEvaluatePolicy. Confirmed by reading the code.

M-10 LOW Silent keys usable while the device is locked

CODE READING

`Android | Plugin.kt:405–416, 955–995`

The docs say "while unlocked" and iOS does that. On Android a background webview can still sign and decrypt with the screen locked.

Fix: Add setUnlockedDeviceRequired(true). Confirmed by reading the code.

M-11 LOW sign() prompts before validating

CODE READING

`iOS | Plugin.swift:411–422`

Face ID is shown before checking that the alias exists or that the key type matches. JS can pull prompts for aliases that do not exist.

Fix: Validate first, then prompt. Confirmed by reading the code.

THE OTHER FINDINGS | 2 OF 2

M-12 to M-18

M-12 LOW setItem limits count characters, not bytes — CODE READING

`iOS | Plugin.swift:1159–1161`

One Swift character can have many code points. A value of 256K characters can take several MB in the Keychain.

Fix: Limit utf8.count. Confirmed by reading the code.

M-13 LOW recipientPublicKey has no cap before decoding — CODE READING

`iOS + Android | Plugin.swift:539–543`

The earlier cap covered only the data field. A public key of many MB is decoded in full before its length is checked.

Fix: Cap the base64 length first. Confirmed by reading the code.

M-14 LOW Secure Enclave signing key is created without a lock — CODE READING

`iOS | Plugin.swift:757–786`

Two concurrent first calls can create two keys and cause random E_TAMPERED errors. Hard to reach in practice.

Fix: Add a static lock. Confirmed by reading the code. Reach is doubtful.

M-15 LOW Backup exclusion in the README is incomplete — CODE READING

`Docs | README.md:241–259`

It lists one preferences file. pq_kem.xml and pq_sig.xml are missing.

Fix: Add them to the snippet. Confirmed by reading the code.

M-16 LOW Key prefix is global state — CODE READING

`TypeScript | enhance.ts:20–37`

setKeyPrefix changes one shared instance. A library that sets it also redirects the host app, and a clear() with no prefix wipes every prefix.

Fix: Add withPrefix() that returns a separate instance. Confirmed by reading the code.

M-17 LOW Caller controls the prompt title on Ed25519 — CODE READING

`Android | Plugin.kt:216, 1049–1053`

For wrapped Ed25519 keys the caller text becomes the prompt title. For hardware keys it is only the subtitle.

Fix: Fixed title, description as subtitle. Confirmed by reading the code.

M-18 LOW Non-boolean flags are treated as defaults — CODE READING

`All | Plugin.kt:1205, Plugin.swift:1177`

setItem({ requireBiometric: "true" }) stores a silent item with no error.

Fix: Reject with E_INVALID_ARGS. Confirmed by reading the code.

HONEST ACCOUNTING

What was checked and dropped.

These suspects were reviewed on purpose and do not count as findings. They are listed so the next round does not repeat them.

- authInFlight leak on Android: every release path traced, no leak.
- Public key tag swap between aliases on iOS: needs root and both keys belong to the same user.
- Tier probe on a locked iOS device: fails closed.
- BouncyCastle addProvider returning -1: harmless, the plugin passes its own provider instance.
- Web fallback: nothing new beyond the documented limits.
- ECDSA low-S handling: iOS, Android and noble agree, including the boundary value.
- Sizes and encodings of keys and signatures: equal on the three platforms.
- CI workflow: no secrets, no pull_request_target, empty default permissions.

What was covered weakly.

- Swift code that depends on the system keychain daemon is reasoned, not tested.
- iOS concurrency: assumes a serial bridge queue. No equivalent of authInFlight, so JS can queue many Face ID prompts.
- Android ML-DSA names in KeyMint were not verified.
- The test app and e2e specs were only skimmed. Supply chain only checked at the version pins.

> **The model reasoned about system behavior it could not run. Those parts are marked plausible or listed as device checks, not as facts.**

AFTER THE AUDIT

All 18 findings fixed and released.

The fixes were applied on 2026-10-30 on a branch from 7.x, one commit per finding, with a failing test first wherever a test can run without a device. The same commits were ported to master. They shipped as **7.1.0** on the Capacitor 7 line and **8.1.0** on the Capacitor 8 line.

52
JS tests (vitest)

30
Android JVM tests

15
iOS tests on the simulator

The Android emulator suite passes 5 of 6 specs, the same as the untouched 7.x. The ML-DSA signing spec fails on any emulator for lack of KeyMint. TESTED: a test covers the changed logic. BUILT: compiles, no direct test. DEVICE: only a phone can confirm. DOCS: documentation.

ID	CHECK	WHAT WAS DONE
M-1	TESTED	Key reuse rule unit-tested, red first. The Keystore trigger still needs a device.
M-2	TESTED	Live-key choice unit-tested. The prompt itself still needs a device.
M-3	TESTED	Known-answer test with noble vectors failed before the fix. Cross-platform run needs devices.
M-4	TESTED	Strict decoder unit-tested. Other methods got a guard that rejects instead of throwing.
M-5	TESTED	Algorithm mapping unit-tested. The name an ML-DSA key reports needs a device.
M-6	BUILT	Wrap key is versioned, alias helper unit-tested. The replace flow needs a device.
M-7	BUILT	Both namespaces checked on Android and iOS. No Keystore or Keychain in unit tests.
M-8	DEVICE	Listing uses UIFail now. The simulator cannot answer, so it stays plausible until a phone does.
M-9	TESTED	biometricGated follows enrollment. Simulator test.
M-10	BUILT	Silent keys need an unlocked device. Emulator suite unchanged, the locked case needs a device.
M-11	TESTED	Unknown alias or wrong type rejects before any prompt. Simulator test.
M-12	TESTED	Limits count UTF-16 units like Android and web. Simulator test with combining marks.
M-13	TESTED	Cap applied before decoding on all platforms. Tested on web and iOS.
M-14	BUILT	Lock added around key creation. Concurrency is not tested.
M-15	DOCS	README lists all three preference files.
M-16	TESTED	withPrefix added, setKeyPrefix deprecated. Covered by vitest.
M-17	BUILT	Fixed title, the caller text is the subtitle. The prompt UI is not tested.
M-18	TESTED	Non-boolean flags reject with E_INVALID_ARGS on web, iOS and Android.

> **One break on purpose: frames sealed on Android with encryptTo by 7.0.0 and 8.0.0 no longer open. They never worked on iOS, web or any other library. The changelog says so.**

STILL OPEN

Eight checks that need a real device.

1. iOS, M-8: with one silent and one biometric item, keys() must list both and clear() must ask for Face ID.
2. iOS: after a Face ID re-enrollment, an old biometric item can still be checked, removed with a prompt, and never overwritten silently.
3. iOS: Ed25519 with requireBiometric and no biometric enrolled. Does the alias keep a public key with no private key?
4. Android, M-1: a failed silent write while locked leaves an orphan key. Then a biometric setItem must not reuse it.
5. Android, M-2: biometric item X, change enrollment, new biometric item Y, clear() must prompt.
6. Android, M-5: which algorithm name does an ML-DSA key report, and does overwrite of an ECDSA key now prompt?
7. Cross, M-3: encryptTo on web to an Android public key, then decrypt on the Android. Both directions.
8. Android: open prompt plus screen rotation. Does authInFlight stay set and block every later call?

Fix order, as applied.

The order in which the fixes were done on the branch from 7.x, then ported to master.

#	FINDING	EFFORT	WHY HERE
1	**M-3**	2 lines + known-answer test	Unblocks cross-platform KEM and pins the format
2	**M-4**	1 function	One call from the webview closes the app
3	**M-1**	3 lines	High: tier downgrade
4	**M-2**	6 lines	High: wipe without a prompt
5	**M-5**	1 function	Bricked ECDSA alias
6	**M-7**	4 places	Shadowing of the identity alias
7	**M-6**	small refactor	Zombie alias after a failed replacement
8	**M-9, M-11, M-12, M-13**	1 to 5 lines each	iOS, cheap
9	**M-8**	10 lines	Safe under both semantics
10	**M-10, M-14 to M-18**	small	Hardening and docs

PLAYBOOK

Run it on your own code.

The steps that made the audit useful, in order.

1	**Pin the code**	One commit, clean tree. Write the hash in the report.
2	**Write the threat model first**	Who attacks, what counts, what does not count.
3	**Give it everything**	Source, tests, docs, CI and the history of earlier findings.
4	**Set strict rules**	File and line, a concrete scenario, a fix, severity, honest confidence.
5	**Hunt with several lenses**	One focus per pass: each lens finds different bugs.
6	**Make it kill its findings**	Second pass with three verdicts: confirmed, plausible, rejected.
7	**Run what can be run**	Small experiments on the JVM or in Node. Keep the output.
8	**Write down the gaps**	What was not covered and what needs a real device.
9	**Fix with a red test first**	One commit per finding.

Versions used

Claude Mythos 5.1 (claude-mythos-5-1) in Claude Code. BouncyCastle bcprov-jdk18on 1.85. @noble/post-quantum 0.6.1. Capacitor Android 8.4.1 (source reading). JDK 24.0.2. Plugin 7.0.0 at commit 8d9a0f4, fixed in 7.1.0 and 8.1.0.

Sources

github.com/jimcase/capacitor-pq-secure-storage

Notice

Independent audit of the author's own open-source plugin, run with Claude Mythos 5.1. Claude, Mythos and the Anthropic logo are trademarks of Anthropic, PBC. This document is not affiliated with, sponsored or endorsed by Anthropic. The findings are the model's analysis plus local checks. Items marked plausible or needing a device are not confirmed.